How R2 Works

Whole-plan repair with regression recording · `chinatravel/agent/nesy_verifier/repair_strategies/r2.py`

> **In one sentence:** R2 asks the LLM to rewrite the **whole plan** against the current failures and keeps whatever usable plan comes back, even one that breaks a constraint that used to pass. A break is *recorded*, not reverted. R2 only falls back to the unchanged plan when it cannot get a usable plan at all (case 4).

1. Where R2 sits in the repair loop

The shared loop (`shared_repair_loop.py::run_repair_loop`) drives every strategy. On each step it:

1. picks a phase, `"personal"` or `"commonsense"`, and passes it in `context["repair_phase"]`;
2. calls `strategy.repair(...)` to get a candidate plan;
3. verifies the candidate with the shared verifier;
4. calls `strategy.accept(current_verification, candidate_verification)`;
5. only then reads `result.metadata` and writes the attempt record.

Because the metadata is read after `accept()`, `accept()` can still add `decision_reason` to it.

Phase schedule (`total_budget = min(max_steps, 6)`): personal steps run while any personal constraint fails, up to 5 attempts. One commonsense step runs once all personal constraints pass or the 5 attempts are used up. If the commonsense step breaks a personal constraint, the loop returns to personal repair, as long as budget and personal attempts remain. For "5 personal + 1 commonsense", run with `--refine-steps 6`.

2. Helper functions

`_failed_personal_targets(verification_results)` — line 21

Reads `personal_constraints`, a list of `True` / `False` / `None` with one entry per `hard_logic_py` constraint. Returns the **1-based** positions of the entries that are exactly `False`.

- `[True, False, None, False]` → `[2, 4]`
- `None` means the verifier could not evaluate that constraint, so it is not a target. R2 only repairs constraints that have definitely failed.

`_protected_personal_indices(verification_results)` — line 31

The opposite: the 1-based positions of entries that are exactly `True`, i.e. the constraints passing right now.

- `[True, False, None, False]` → `[1]`
- Used in the metadata (`protected_constraint_indices`) and in the prompt's "keep these satisfied" list. They do **not** block a candidate.

`_collect_target_feedback(verification_results, targets)` — line 41

Picks the messages in `error_info` that start with `"Personal constraint {k} failed:"` for each target `k`. Commonsense messages are left out, so a personal-phase prompt only mentions personal constraints. The colon in the prefix stops `"Personal constraint 1 failed:"` from wrongly matching constraint 10.

`_collect_commonsense_feedback(verification_results)` — line 58

Returns the `commonsense_errors` list, keeping only strings. Used only in the commonsense phase, e.g. "*The end_time of dinner … cannot be earlier than 17:00.*"

`_frame_feedback(problem, feedback, protected)` — line 68

Builds the "There are some concerns:" section of the prompt in three parts:

1. A header stating that the plan **violates** the requirements below, that returning it unchanged is not acceptable, and listing example changes (cheaper transport, a different hotel, other times).
2. The feedback messages.
3. If any constraints pass: "While fixing these, keep the following satisfied:", followed by each one's DSL code from `hard_logic_py`.

The header exists because in the pilot5 run the model returned the plan unchanged on all 15 calls.

`_build_plan_prompt(problem, raw_plan, context, feedback)` — line 96

Fills in ChinaTravel's existing `BACK_PROMPT` template with:

- **from `context`:** the sandbox data (train and flight options, attractions, restaurants, hotels, inner-city transport);
- **from `problem`:** the user's request in natural language;
- **the current plan**, as JSON;
- **the feedback lines**, one per line.

The template ends with `Fixed Travel Plan (…):\n[{\n "day": 1,`, a partial JSON opening the model is meant to continue.

`_is_usable_candidate(candidate)` — line 119

Checks only the **shape** of a parsed plan, not whether it satisfies any constraint. It must be a non-empty list; every day must be a dict with an `activities` list; every activity must be a dict; and at least one day must have at least one activity. Anything else becomes case 4.

`_parse_candidate_response(response)` — line 139

Turns the model's text into a plan. It tries up to two readings:

1. **The fenced block:** if the reply contains ```` ```json … ``` ```` (or an unclosed fence), it parses what is inside. DeepSeek on the chat API actually answers this way.
2. **The prefilled framing:** `[{"day": 1,` + the response, which is how the template expects the model to reply.

Each reading goes through `repair_json` (fixes trailing commas, missing brackets and similar). A single-day object `{…}` is wrapped as `[{…}]`. The first reading that passes `_is_usable_candidate` wins. If neither does, it raises `ValueError`, which `repair()` turns into case 4. It always returns the framed text too, which is logged as `raw_response`.

`_has_protected_regression(current_verification, candidate_verification)` — line 166

Compares the personal-constraint results before and after the rewrite, position by position. Returns `True` if any constraint went from `True` to `False`.

- Changes involving `None` (e.g. `True → None`) do not count as a regression.
- If the two lists have different lengths, it returns `False`, because they cannot be compared position by position.

3. The `R2ProtectedRepair` class — line 179

The name is historical. The class no longer rolls anything back.

`__init__` — line 187

Creates `self._last_result`, which holds the result of the most recent `repair()` call. `accept()` needs it, because the loop passes `accept()` only the two verification results, not the result object.

`repair(problem, raw_plan, verification_results, context, llm)` — line 192

Stage	Lines	What happens
1. Targets	204–212	Checks whether `context["repair_phase"]` is `"commonsense"`. Commonsense phase: `targets = []`. Personal phase: the failed personal constraints. `protected` is always the passing ones. Starts the metadata with `strategy`, `target_constraint_indices`, `editable_scope: "plan"` and `protected_constraint_indices`.
2–3. Feedback	217–225	Commonsense phase → commonsense errors. Personal phase → messages for the targets. If there is no feedback, it returns **a copy of the unchanged plan** with `failure_reason: no_valid_repair_candidate` (case 4), without calling the model.
4. Prompt	229–231	`_frame_feedback`, then `_build_plan_prompt`.
5. Model call and parsing	235–250	Calls `llm(...)`, times it, and runs `_parse_candidate_response`. Any exception (API error, unparseable output, wrong shape) gives case 4: the unchanged plan, `failure_reason`, and a debug `candidate_error`.
6. Success	254–262	Adds `feedback_personal_constraint_indices` (the targets; empty in the commonsense phase) plus debug fields `prompt`, `raw_response` and `llm_inference_time`, the last of which the agent adds to its reported inference time. Returns the **candidate plan** (cases 1–3).

`repair()` never checks whether the candidate actually fixes anything. That is the loop verifier's job, which is why cases 1, 2 and 3 look the same at this point.

`accept(current_verification, candidate_verification)` — line 264

Called by the loop after it has verified the candidate. It runs `_has_protected_regression`, then **always returns `True`**, so the candidate becomes the current plan even if something regressed. The regression check only decides whether `decision_reason` gets recorded.

`_record_decision(accepted, *, regressed)` — line 279

Edits the metadata of `self._last_result`. If something regressed, it sets `decision_reason: "previously_satisfied_constraint_regressed"` (case 2). Otherwise it removes any leftover `decision_reason`, in case the same strategy object is reused across attempts. It then returns `accepted` unchanged, which is always `True` here.

4. How the four cases come out

Case	Path through the code	Plan returned	Extra metadata
1. Target fixed, nothing broken	stage 6 → `accept`, no regression	candidate	none
2. Target fixed, a passing constraint broken	stage 6 → `accept`, regression	candidate	`decision_reason`
3. Target not fixed, nothing broken	stage 6 → `accept`, no regression	candidate	none
4. No usable candidate	stage 2–3 (no feedback) or stage 5 (exception)	unchanged copy	`failure_reason`

Example metadata for case 2:

```
{
    "target_constraint_indices": [2],
    "editable_scope": "plan",
    "protected_constraint_indices": [1, 3, 4],
    "feedback_personal_constraint_indices": [2],
    "decision_reason": "previously_satisfied_constraint_regressed"
}
```

Several failed constraints are handled in one call: `targets` can be `[2, 3]`, and both messages go into the same prompt.

5. Known limitations

- **Only one commonsense step.** If the commonsense rewrite creates *new* commonsense errors while all personal constraints pass, the loop stops even with budget left. Changing this needs an edit to `shared_repair_loop.py`, which affects every strategy.
- **The 5-attempt personal cap also limits recovery.** After commonsense, personal recovery only runs if fewer than 5 personal attempts have been used.
- **English runs have a label mismatch.** Some English queries check Chinese category labels (e.g. `'川菜'`, `'自然风光'`) while the English sandbox returns English labels (`"Sichuan cuisine"`, `"natural scenery"`). No plan can satisfy those constraints, for any strategy.
- **Feedback is raw DSL code.** Personal-constraint messages show the constraint's code, not computed values such as the plan's actual total cost.